

Laporan Final Project

Natural Language Processing



**Klasifikasi Email Spam dan Non-Spam Menggunakan Pendekatan
Text Classification Berbasis Machine Learning**

Nama Tim Kelompok 9 :

1. Ercent Tannius - 2802443565
2. Nicholas Lee - 2802450721
3. Jos Hwen Sim - 2802451655
4. Winson Jonathan - 2802446730
5. Yohanes Kevin Lie - 2802444454

**COMP6885001 - Natural Language Processing Even Semester
2025/2026**

Bina Nusantara University

Abstract

Spam email tetap menjadi salah satu ancaman paling gigih dalam infrastruktur komunikasi digital, dengan lebih dari 45% lalu lintas email global diklasifikasikan sebagai email yang tidak diminta atau berbahaya. Proyek ini menyajikan alur kerja klasifikasi teks ujung-ke-ujung untuk deteksi spam yang memanfaatkan Multinomial Naive Bayes yang dikombinasikan dengan representasi fitur TF-IDF dan teknik pra-pemrosesan Pemrosesan Bahasa Alami yang diimplementasikan melalui NLTK dan scikit-learn. Alur kerja pra-pemrosesan mencakup normalisasi huruf kecil, penghapusan noise berbasis regex, penghapusan stopword, dan Porter Stemming untuk mengurangi dimensi leksikal dan meningkatkan konsistensi fitur. Vektorisasi TF-IDF dikonfigurasi dengan `max_features=5000`, `ngram_range=(1,2)`, `min_df=2`, dan `max_df=0.8` untuk menangkap pola unigram dan bigram yang menjadi ciri khas konten spam. Selain itu, enam fitur heuristik diekstrak dari korpus email mentah, termasuk panjang pesan, jumlah kata, deteksi simbol mata uang, konten numerik, karakter khusus, dan indikator kosakata urgensi, meskipun fitur-fitur ini tidak diintegrasikan ke dalam fase pelatihan model akhir karena adanya kesenjangan implementasi. Model tersebut mencapai akurasi 98,26%, presisi 94,03%, recall 92,65%, dan skor F1 93,33% pada set pengujian yang dipisahkan sebesar 20%. Hasil menunjukkan bahwa pengklasifikasi probabilistik yang disetel dengan baik dan diproses secara teliti tetap kompetitif untuk tugas deteksi spam. Keterbatasan yang teridentifikasi meliputi asumsi independensi yang melekat pada Naive Bayes, kerentanan terhadap teknik penghindaran substitusi karakter, dan kumpulan fitur heuristik yang tidak dimanfaatkan, yang semuanya merupakan arahan yang jelas untuk peningkatan di masa mendatang menuju sistem penyaringan spam yang lebih kuat dan adaptif.

Kata kunci: deteksi spam, Multinomial Naive Bayes, TF-IDF, klasifikasi teks, Pemrosesan Bahasa Alami, rekayasa fitur

1. Pendahuluan

1.1. Latar Belakang

Email telah menjadi tulang punggung komunikasi digital, namun dibalik itu spam email tumbuh sebagai ancaman yang tidak bisa diabaikan. Lebih dari 45% seluruh lalu lintas email global dikategorikan sebagai spam, mulai dari iklan tak diundang hingga phishing yang menarget kredensial pengguna dan menyebarkan malware. Kerugian akibat serangan siber berbasis email diperkirakan mencapai miliaran dolar per tahun secara global, dengan phishing menjadi vektor serangan yang paling banyak dilaporkan dalam beberapa tahun terakhir sehingga ancaman ini tidak lagi bisa dianggap sekadar gangguan kecil.

Pendekatan rule-based filtering terbukti mudah dielakkan spammer yang terus berinovasi, sehingga Machine Learning hadir sebagai solusi adaptif yang mampu belajar dari pola data secara otomatis. Di antara berbagai algoritma yang tersedia, Multinomial Naive Bayes menonjol karena interpretabilitasnya tinggi, training cepat, dan terbukti efektif untuk klasifikasi teks berbasis probabilistik. Proyek ini mengintegrasikan Naive Bayes dengan teknik NLP menggunakan NLTK dan scikit-learn, membentuk pipeline

end-to-end yang mencakup prapemrosesan, feature engineering, hingga penyimpanan model yang siap digunakan.

1.2. Rumusan Masalah

- Bagaimana membangun pipeline klasifikasi email spam menggunakan Multinomial Naive Bayes secara end-to-end?
- Bagaimana melakukan prapemrosesan dan feature engineering pada dataset email.csv agar optimal untuk klasifikasi?
- Seberapa tinggi performa model Naive Bayes (akurasi, presisi, recall, F1-score) pada dataset yang digunakan?
- Apa saja kelebihan dan keterbatasan pendekatan Naive Bayes untuk kasus ini?

1.3. Tujuan Penelitian

- Memahami dan mengimplementasikan algoritma Multinomial Naive Bayes untuk menyelesaikan permasalahan klasifikasi teks, khususnya deteksi email spam.
- Menerapkan teknik prapemrosesan teks menggunakan library NLTK, meliputi tokenisasi, stopword removal, dan Porter Stemming.
- Merepresentasikan data teks sebagai fitur numerik menggunakan metode TF-IDF serta mengekstraksi fitur tambahan berbasis karakteristik email spam.
- Melatih model Naive Bayes dan mengevaluasi performanya menggunakan metrik akurasi, presisi, recall, dan F1-score serta confusion matrix.

1.4. Signifikansi

Secara akademis, proyek ini mendemonstrasikan penerapan NLP probabilistik pada klasifikasi teks. Secara praktis, model yang tersimpan langsung dapat dihubungkan ke layanan email atau sistem keamanan siber tanpa perlu pelatihan ulang.

2. Tinjauan Pustaka

2.1. Sejarah dan Perkembangan Deteksi Spam

Penelitian deteksi spam berkembang sejak awal 2000-an, dimulai dari rule-based filtering yang mudah ditembus spammer. Graham (2002) memperkenalkan Bayesian spam filtering yang kemudian menjadi fondasi sistem modern seperti SpamAssassin. Perkembangan selanjutnya mengintegrasikan Machine Learning untuk menangkap pola yang lebih kompleks.

2.2. Algoritma Naive Bayes untuk Klasifikasi teks

Naive Bayes adalah algoritma probabilistik yang menerapkan Teorema Bayes dengan asumsi independensi antar fitur. Meskipun asumsi ini jarang terpenuhi dalam data nyata, algoritma ini menunjukkan performa sangat baik untuk klasifikasi teks (Androutsopoulos et al., 2000). Tiga varian utamanya adalah Bernoulli Naive Bayes untuk data biner, Multinomial Naive Bayes untuk klasifikasi teks dengan fitur frekuensi kata atau TF-IDF, dan Gaussian Naive Bayes untuk fitur kontinu berdistribusi Gaussian. McCallum dan Nigam (1998) menyimpulkan varian Multinomial secara konsisten unggul pada dataset berbasis teks dibanding dua varian lainnya.

2.3. TF-IDF dan Feature Engineering

TF-IDF (Term Frequency-Inverse Document Frequency) memberi bobot lebih tinggi pada kata yang sering muncul dalam dokumen tertentu tetapi jarang di seluruh korpus, sehingga lebih informatif dibanding raw word count (Pedregosa et al., 2011). Parameter `max_df` mengabaikan kata yang terlalu umum di seluruh koleksi, sedangkan `min_df` mengabaikan kata yang sangat jarang muncul. Selain fitur teks, penelitian sebelumnya menunjukkan bahwa karakteristik struktural email seperti keberadaan simbol mata uang, angka, karakter khusus, dan kata-kata urgensi berkorelasi kuat dengan label spam dan dapat meningkatkan performa klasifikasi secara signifikan (Metsis et al., 2006).

2.4. Solusi yang Ada

Beberapa solusi deteksi spam yang telah dikembangkan antara lain SpamAssassin yang mengombinasikan aturan heuristik dengan skor Bayesian, serta sistem filter Gmail yang menggunakan kombinasi Machine Learning dengan akurasi lebih dari 99%. Dibandingkan pendekatan deep learning seperti LSTM dan BERT, Naive Bayes menawarkan keunggulan dalam interpretabilitas, kecepatan training, dan kebutuhan data yang lebih sedikit, menjadikannya pilihan ideal untuk proyek dengan sumber daya terbatas (Metsis et al., 2006).

3. Metodologi

3.1. Dataset

Dataset yang digunakan adalah Spam Email Classification yang bersumber dari [spam-email-dataset](#), memiliki dua kolom utama: Category (label kelas: 'ham' atau 'spam') dan Message (isi teks email). Dataset dibaca menggunakan pandas dengan encoding latin-1 untuk menangani karakter khusus. Pembersihan awal dilakukan dengan memfilter label non-standar dan menghapus duplikat via `drop_duplicates()` untuk mencegah data leakage.

3.2. Prapemrosesan Data

Fungsi `clean_text()` menerapkan pipeline pembersihan secara berurutan: lowercasing untuk normalisasi kapitalisasi, penghapusan karakter non-alfabet via regex, normalisasi whitespace, tokenisasi dan stopwords removal menggunakan daftar Bahasa Inggris dari NLTK, lalu Porter Stemming untuk mereduksi kata ke bentuk dasarnya, misalnya 'running' menjadi 'run' dan 'prizes' menjadi 'prize'. Fungsi ini diaplikasikan ke seluruh kolom Message menggunakan `apply()` dan hasilnya disimpan ke kolom baru `cleaned_message` yang menjadi input untuk tahap vektorisasi. Kode lengkap tersedia di Lampiran B.1.

3.3. Feature Engineering

Enam fitur tambahan diekstraksi dari kolom Message asli sebelum proses cleaning untuk menangkap karakteristik spam yang bersifat struktural. Fitur `message_length` dihitung menggunakan `len(Message)` karena email spam cenderung lebih panjang, sedangkan `word_count` diperoleh dari `len(Message.split())` sebagai proksi panjang pesan. Tiga fitur biner diekstraksi via regex yaitu `has_currency` untuk mendeteksi simbol mata uang seperti \$, £, €, dan ¥ yang sering muncul di spam promosi, `has_numbers` untuk mendeteksi angka yang umumnya berupa nomor telepon atau kode promo, serta `has_special_chars` untuk mendeteksi karakter seperti !, @, dan # yang penggunaannya berlebih khas spam. Fitur terakhir `has_urgent_words` mendeteksi kata-kata urgensi seperti 'free', 'win', 'prize', 'winner', 'cash', dan 'guarantee' yang mendominasi konten spam. Pada implementasi final keenam fitur ini belum diintegrasikan ke model karena X hanya menggunakan matriks TF-IDF, dan hal ini dibahas lebih lanjut sebagai limitasi di Bab 5.

3.4. Label Encoding dan Vektorisasi

Kolom Category dikonversi ke nilai numerik dengan `ham=0` dan `spam=1`. Teks yang telah dibersihkan kemudian divektorisasi menggunakan `TfidfVectorizer` dengan konfigurasi `max_features=5000` untuk mengambil 5.000 term tertinggi, `ngram_range=(1,2)` untuk menangkap unigram dan bigram seperti 'call now' atau 'free prize', `min_df=2` untuk mengabaikan term yang sangat jarang, dan `max_df=0.8` untuk mengabaikan term yang terlalu umum. Kode lengkap tersedia di Lampiran B.2.

3.5. Pembagian Data

Data dibagi dengan rasio 80% training dan 20% testing menggunakan `train_test_split` dengan `random_state=42` untuk reproduibilitas. Fitur input X adalah matriks TF-IDF dan label y adalah kolom numerik hasil encoding.

3.6. Arsitektur Model : Multinomial Naive Bayes

Model yang digunakan adalah MultinomialNB dari scikit-learn dengan parameter $\alpha=0.1$. Nilai α yang lebih kecil dari standar Laplace ($\alpha=1.0$) berarti smoothing lebih ringan sehingga model lebih sensitif terhadap distribusi probabilitas data training. Dasar matematis model mengikuti Teorema Bayes:

$$P(C|X) = \frac{P(X|C) * P(C)}{P(X)}$$

di mana C adalah kelas (ham/spam) dan X adalah vektor fitur dokumen.

3.7. Metrik Evaluasi

Metrik	Formula	Keterangan
Accuracy	$\frac{TP + TN}{Total}$	Proporsi prediksi benar dari keseluruhan
Precision	$\frac{TP}{TP + FP}$	Dari prediksi spam, berapa yang benar spam
Recall	$\frac{TP}{TP + FN}$	Dari semua spam, berapa yang terdeteksi
F1-Score	$2 * \frac{P * R}{P + R}$	Harmonik mean precision dan recall

4. Result and Discussion

4.1. Hasil

4.1.1. Interpretasi Hasil Model

Model Multinomial Naive Bayes yang dibangun berhasil mencapai akurasi 98.26%, presisi 94.03%, recall 92.65%, dan F1-score 93.33% pada data uji dengan rasio pembagian 80:20. Confusion matrix mengonfirmasi 888 true negative, 126 true positive, 8 false positive, dan hanya 10 false negative, menunjukkan model cukup andal menangkap spam meski masih ada ruang perbaikan pada sisi recall.

4.2. Diskusi

4.2.1. Kontribusi Pipeline Pemrosesan Teks

Pipeline prapemrosesan memainkan peran krusial yang setara dengan pemilihan algoritma itu sendiri. Lowercasing menghilangkan ambiguitas kapitalisasi, regex cleaning menyingkirkan noise non-alfabet, dan stopword removal mereduksi dimensi fitur secara signifikan sehingga model lebih fokus pada kata-kata diskriminatif. Porter Stemming mengurangi sparsity matriks TF-IDF dengan menyatukan berbagai bentuk infleksi ke satu representasi dasar. Konfigurasi TF-IDF dengan `ngram_range=(1,2)` terbukti efektif karena bigram mampu menangkap frasa khas spam seperti "click here" dan "act now", sementara `min_df=2` dan `max_df=0.8` menyaring term yang terlalu jarang maupun terlalu umum.

4.2.2. Peran Feature Engineering Heuristik

Enam fitur heuristik diekstraksi untuk menangkap karakteristik struktural spam: `message_length`, `word_count`, `has_currency`, `has_numbers`, `has_special_chars`, dan `has_urgent_words`. Masing-masing dirancang berdasarkan pola empiris konten spam seperti penggunaan simbol mata uang, angka promo, karakter khusus berlebih, dan kosakata urgensi. Namun pada implementasi final, variabel `X` hanya menggunakan matriks TF-IDF tanpa mengintegrasikan keenam fitur ini, sehingga seluruh informasi struktural yang berpotensi meningkatkan recall menjadi tidak dimanfaatkan. Ini adalah celah implementasi yang perlu diperbaiki pada iterasi berikutnya.

4.3. Limitasi

4.3.1. Fitur Heuristik Tidak Diintegrasikan ke dalam Model

Keenam fitur heuristik yang sudah diekstraksi tidak ikut dilatih bersama model karena `X` hanya berisi matriks TF-IDF. Solusinya adalah menggabungkan keduanya via `scipy.sparse.hstack()` setelah fitur heuristik dikonversi ke array numerik, yang diperkirakan meningkatkan recall terutama pada spam berkarakter khusus dengan kosakata tidak umum.

4.3.2. Asumsi Independensi Fitur yang Tidak Realistis

Naive Bayes mengasumsikan setiap fitur independen secara kondisional, padahal dalam bahasa alami kata-kata memiliki ketergantungan kontekstual yang kuat, contohnya "click" dan "here" hampir selalu muncul bersama dalam konteks spam. Bigram

ngram_range=(1,2) hanya menangkap ketergantungan pada pasangan kata yang berdekatan, sedangkan long-range dependencies sama sekali tidak terjangkau. Model berbasis transformer seperti BERT dirancang mengatasi hal ini melalui mekanisme attention dan terbukti mencapai recall hingga 99% pada deteksi spam.

4.3.3. Kerentanan terhadap Teknik Inovasi Spam

Model sangat bergantung pada pencocokan kata dalam representasi bag-of-words, sehingga rentan terhadap teknik evasion modern seperti substitusi karakter ("v1agra"), penyisipan teks acak, pengiriman spam berbasis gambar, dan personalisasi konten. Substitusi karakter sederhana sudah cukup menjatuhkan performa karena kata hasil manipulasi tidak terdapat dalam kosakata pelatihan. Sifat model yang statis memperparah kerentanan ini di tengah lanskap spam yang terus berevolusi.

5. Kesimpulan dan Saran

5.1. Kesimpulan

Proyek ini membuktikan bahwa Multinomial Naive Bayes berbasis TF-IDF, bila dikombinasikan dengan pipeline prapemrosesan yang terstruktur, tetap kompetitif sebagai solusi deteksi spam. Model mencapai akurasi 98.26%, presisi 94.03%, recall 92.65%, dan F1-score 93.33%, hasil yang solid untuk arsitektur probabilistik ringan seperti ini. Pipeline lowercasing, stopword removal, regex cleaning, dan Porter Stemming terbukti sinergis membentuk representasi teks yang bersih, sementara konfigurasi bigram pada TF-IDF berhasil menangkap frasa diskriminatif seperti "click here" dan "free prize" yang tidak tertangkap unigram.

Namun proyek ini menyisakan satu celah yang disayangkan: enam fitur heuristik (message_length, word_count, has_currency, has_numbers, has_special_chars, has_urgent_words) yang berhasil diekstraksi justru tidak diintegrasikan ke dalam pelatihan model karena variabel X hanya memuat matriks TF-IDF, sehingga potensi peningkatan recall terbuang sia-sia. Dari sisi teoritis, asumsi independensi kondisional Naive Bayes memang tidak terpenuhi dalam bahasa alami [2][3], namun model tetap bekerja baik karena distribusi kosakata spam dan ham secara empiris berbeda secara konsisten. Pemilihan $\alpha=0.1$ juga terbukti tepat karena memberikan sensitivitas lebih tinggi terhadap pola pelatihan dibanding Laplace smoothing standar.

5.2. Saran

Prioritas utama iterasi berikutnya adalah mengintegrasikan fitur heuristik ke dalam matriks fitur via `scipy.sparse.hstack()`, langkah yang relatif mudah namun berpotensi mendongkrak recall secara signifikan terutama pada spam berkarakter khusus dengan kosakata tidak umum. Selain itu, perbandingan dengan Logistic Regression, SVM, dan Random Forest perlu dilakukan untuk mendapatkan gambaran trade-off performa yang lebih objektif pada dataset yang sama.

Untuk ketahanan terhadap evasion modern seperti substitusi karakter dan penyisipan teks acak, penambahan character n-grams direkomendasikan sebagai lapisan fitur yang lebih tahan manipulasi tipografi. Jangka panjang, model berbasis transformer seperti DistilBERT layak dipertimbangkan karena kemampuannya menangkap long-range dependencies yang sama sekali tidak terjangkau bag-of-words, dengan bukti recall hingga 99% pada studi serupa. Terakhir, dataset perlu diperbarui secara berkala dengan sampel spam kontemporer guna mencegah concept drift mengingat lanskap spam berevolusi jauh lebih cepat dari siklus pelatihan ulang model.

Daftar Pustaka

- [1] T. Graham, "A plan for spam," Aug. 2002. [Online]. Available: <http://www.paulgraham.com/spam.html>
- [2] I. Androustopoulos, J. Koutsias, K. V. Chandrinou, G. Paliouras, and C. D. Spyropoulos, "An evaluation of Naive Bayesian anti-spam filtering," in *Proc. Machine Learning in the New Information Age Workshop, ECML 2000*, pp. 9–17.
- [3] A. McCallum and K. Nigam, "A comparison of event models for Naive Bayes text classification," in *AAAI/ICML-98 Workshop on Learning for Text Categorization*, 1998, pp. 41–48.
- [4] V. Metsis, I. Androustopoulos, and G. Paliouras, "Spam filtering with Naive Bayes – Which Naive Bayes?," in *Proc. CEAS 2006*, vol. 17, pp. 28–69.
- [5] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [6] A. Yeafi, "Spam Email Classification Dataset," Kaggle, 2023. [Online]. Available: <https://www.kaggle.com/datasets/ashfakyeafi/spam-email-classification/data>
- [7] B. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python*. O'Reilly Media, 2009.
- [8] Symantec, "Internet Security Threat Report," Broadcom Inc., 2023.
- [9] V. S. Tida and S. Hsu, "Universal spam detection using transfer learning of BERT model," arXiv preprint arXiv:2202.03480, 2022. [Online]. Available: <https://arxiv.org/abs/2202.03480>
- [10] S. Jamal, M. V. Cruz, and J. Kim, "An improved transformer-based model for detecting

phishing, spam and ham emails: A large language model approach," Security and Privacy, Wiley Online Library, 2024. doi: 10.1002/spy2.402

[11] A. B. Ahmed and K. Haruna, "Enhanced SMS spam detection using Bernoulli Naive Bayes with TF-IDF," FU DMA Journal of Sciences, vol. 9, no. 1, pp. 393–399, 2025.

[12] W. Alshanik, R. Bhatnagar, J. Rawashdeh, and A. Abuhamdah, "A deep learning-based spam detection system using domain-specific stop words removal," arXiv:2012.02294, 2020.

[13] T. Ma et al., "A comparative approach to Naive Bayes classifier and support vector machine for email spam classification," in Proc. ICECCME, 2023. doi: 10.1109/ICECCME57830.2023.10252420

[14] S. N. Sari, "Improved spam email detection performance based on Naive Bayes approach TF-IDF vectorizer with multi-metric optimization," Journal of Artificial Intelligence and Engineering Applications (JAIEA), 2025. [Online]. Available: <https://ioinformatic.org/index.php/JAIEA/article/view/981>

Appendix (Lampiran)

Lampiran A - Kontribusi Anggota Tim

1. Ercent Tannius - Train AI, Frontend, Laporan Bab 1-2
2. Jos Hwen Sim - Backend, Frontend, Abstract, Laporan Bab 5
3. Nicholas Lee - Ide Project, Laporan Bab 3
4. Winson Jonathan - Ide Project, Laporan Bab 4 (Implementasi & Hasil)
5. Yohanes Kevin Lie - Integrasi Frontend + Backend + Model, Laporan Bab 5 (Diskusi & Limitasi)

Kami menyatakan bahwa pembagian kontribusi di atas adalah benar dan dilakukan secara adil oleh seluruh anggota kelompok.

Lampiran B - Full Code (AOL.ipynb)

B.1 Import Library dan Load Data

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import re
import nltk
import joblib
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, classification_report

df = pd.read_csv('email.csv', encoding='latin-1')
print("Dataset Shape:", df.shape)
df.isnull().sum()
df.duplicated().sum()

print("=== LABEL CATEGORY Percentase ===")
label_counts = df['Category'].value_counts()
label_percentages = df['Category'].value_counts(normalize=True) * 100
label_summary = pd.DataFrame({'Count': label_counts, 'Percentage': label_percentages})
print(label_summary)

```

B.2. Pembersihan Data dan Prapemrosesan Teks

```

df = df[df['Category'].isin(['ham', 'spam'])]
df = df.drop_duplicates()
nltk.download('stopwords')
stop_words = set(stopwords.words('english'))
stemmer = PorterStemmer()
def clean_text(text):
    text = text.lower()
    text = re.sub(r'^a-zA-Z\s', '', text)
    text = re.sub(r'\s+', ' ', text).strip()
    words = text.split()
    words = [word for word in words if word not in stop_words]
    words = [stemmer.stem(word) for word in words]
    return ' '.join(words)
df['cleaned_message'] = df['Message'].apply(clean_text)

```

B.3 Feature Engineering dan Label Encoding

```

df['message_length'] = df['Message'].apply(len)
df['word_count'] = df['Message'].apply(lambda x: len(x.split()))
df['has_currency'] = df['Message'].apply(lambda x: 1 if re.search(r'[$€¥]', x) else 0)
df['has_numbers'] = df['Message'].apply(lambda x: 1 if re.search(r'\d', x) else 0)
df['has_special_chars'] = df['Message'].apply(lambda x: 1 if re.search(r'[!@#%&*()]', x) else 0)
df['has_urgent_words'] = df['Message'].apply(lambda x: 1 if re.search(r'\b(urgent|free|prize|winner|cash|guarantee)\b', x.lower()) else 0)
df['label'] = df['Category'].map({'ham': 0, 'spam': 1})

```

B.4 Vektorisasi, Training, dan Evaluasi

```

tfidf = TfidfVectorizer(max_features=5000, ngram_range=(1, 2), min_df=2, max_df=0.8)
encoded = tfidf.fit_transform(df['cleaned_message'])
X = encoded
y = df['label']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

nb_model = MultinomialNB(alpha=0.1)
nb_model.fit(X_train, y_train)
y_pred_nb = nb_model.predict(X_test)

print(f"Accuracy : {accuracy_score(y_test, y_pred_nb):.4f}")
print(f"Precision: {precision_score(y_test, y_pred_nb):.4f}")
print(f"Recall   : {recall_score(y_test, y_pred_nb):.4f}")
print(f"F1 score : {f1_score(y_test, y_pred_nb):.4f}")

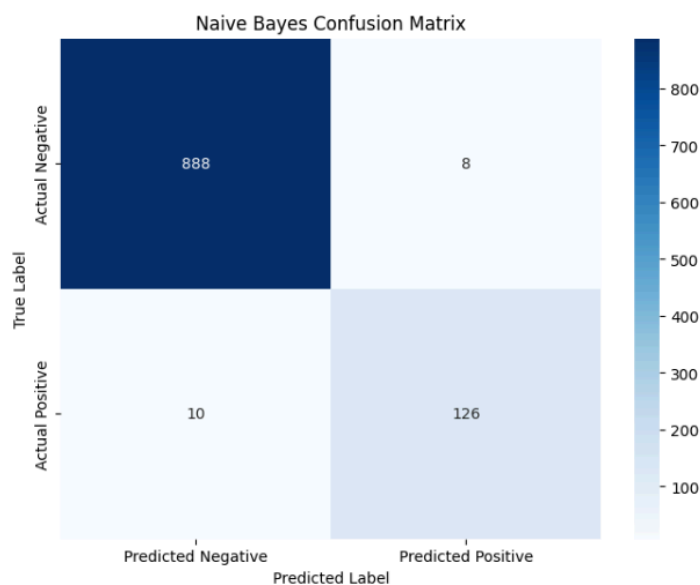
cm = confusion_matrix(y_test, y_pred_nb)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Predicted Negative', 'Predicted Positive'],
            yticklabels=['Actual Negative', 'Actual Positive'])
plt.title('Naive Bayes Confusion Matrix')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.show()

joblib.dump(nb_model, 'spam_model.pkl')
joblib.dump(tfidf, 'tfidf_vectorizer.pkl')

```

Lampiran C - Screenshoot Visualisasi

C.1. Confusion Matrix



C.2. Nilai Metrix

Accuracy: 0.9826
 Precision: 0.9403
 Recall: 0.9265
 F1 score: 0.9333

C.3. Hasil Model

 spam_model.pkl tfidf_vectorizer.pkl

Kode Lengkap Tersedia di : <https://github.com/ercenttannius123/Email-Spam-Detection>